

```
In [8]: # A/B Testing Marketing Campaign
# Full Analysis: Conversion Lift, Revenue Impact, Ad Fatigue, SMOTE, SHAP

# 📦 Required Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score
from imblearn.over_sampling import SMOTE
import shap

# 📄 Load Dataset
df = pd.read_csv('marketing_AB.csv')

# 🔄 Preprocess
df['converted'] = df['converted'].astype(int)
df = pd.get_dummies(df, columns=['test group', 'most ads day'], drop_first=True)

X = df[['total ads', 'most ads hour'] +
       [col for col in df.columns if col.startswith('test group_') or col.startswith('most ads day_')]]
y = df['converted']

X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, random_state=42)

# ⚖️ Apply SMOTE to balance
sm = SMOTE(random_state=42)
X_resampled, y_resampled = sm.fit_resample(X_train, y_train)

# 🌲 Train Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_resampled, y_resampled)

# 📊 Classification Report
y_pred = rf.predict(X_test)
print(classification_report(y_test, y_pred))
print("ROC AUC:", roc_auc_score(y_test, rf.predict_proba(X_test)[:,-1]))

# 📈 Threshold Tuning for Precision-Recall Tradeoff
from sklearn.metrics import precision_recall_curve

# Predict probabilities
y_probs = rf.predict_proba(X_test)[:,-1]

# Calculate precision-recall curve
precisions, recalls, thresholds = precision_recall_curve(y_test, y_probs)

# Plot precision-recall vs. threshold
plt.figure(figsize=(10, 6))
plt.plot(thresholds, precisions[:-1], label='Precision')
plt.plot(thresholds, recalls[:-1], label='Recall')
plt.axvline(x=0.5, color='gray', linestyle='--', label='Default Threshold (0.5)')
plt.xlabel('Threshold')
plt.ylabel('Score')
plt.title('Precision-Recall Tradeoff Curve')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# 📊 Feature Importances
importances = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
importances.plot(kind='barh', figsize=(10, 6), title='Feature Importances')
plt.gca().invert_yaxis()
plt.show()

# 📊 SHAP Analysis
explainer = shap.TreeExplainer(rf)
shap_values = explainer.shap_values(X_test[:1000]) # Limit for speed

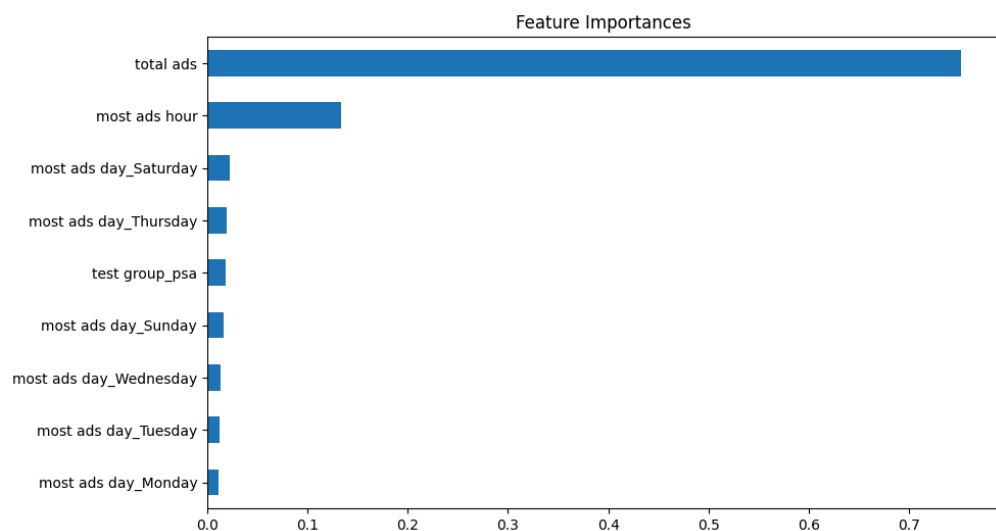
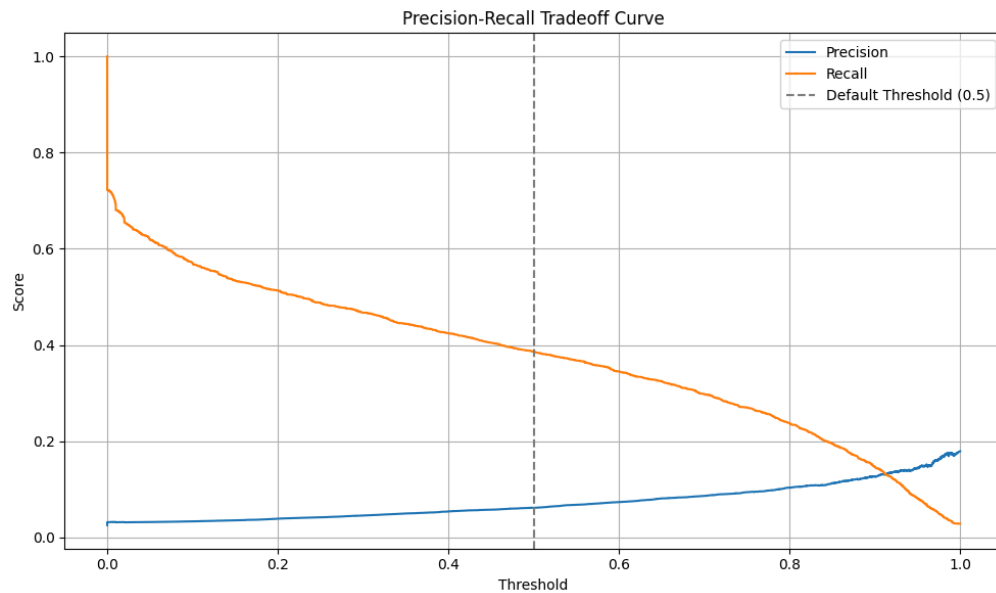
# 🛡️ Safely handle binary classifier SHAP output
if isinstance(shap_values, list) and len(shap_values) == 2:
    shap_values_to_plot = shap_values[1]
else:
    shap_values_to_plot = shap_values # fallback

shap.summary_plot(shap_values_to_plot, X_test[:1000], plot_type='bar')

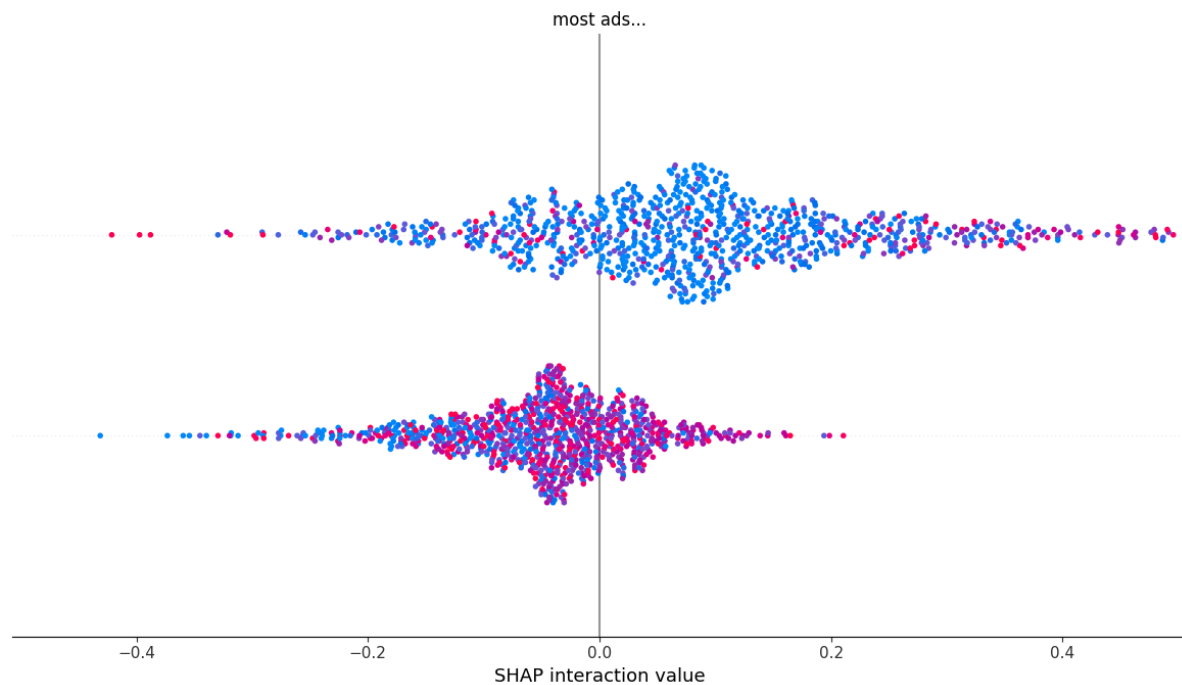
# 📊 Heatmap: Conversion by Day and Hour
original_df = pd.read_csv('marketing_AB.csv')
heat_data = original_df[original_df['test group'] == 'ad'].groupby(['most ads day', 'most ads hour'])['converted'].mean().unstack()
plt.figure(figsize=(16, 10))
sns.heatmap(heat_data, cmap='YlGnBu', annot=True, fmt=".2%")
plt.title('Conversion Rate Heatmap by Day and Hour (Ad Group)')
plt.xlabel('Hour of Day')
plt.ylabel('Day of Week')
plt.tight_layout()
plt.show()
```

	precision	recall	f1-score	support
0	0.98	0.85	0.91	143315
1	0.06	0.39	0.11	3711
accuracy			0.84	147026
macro avg	0.52	0.62	0.51	147026
weighted avg	0.96	0.84	0.89	147026

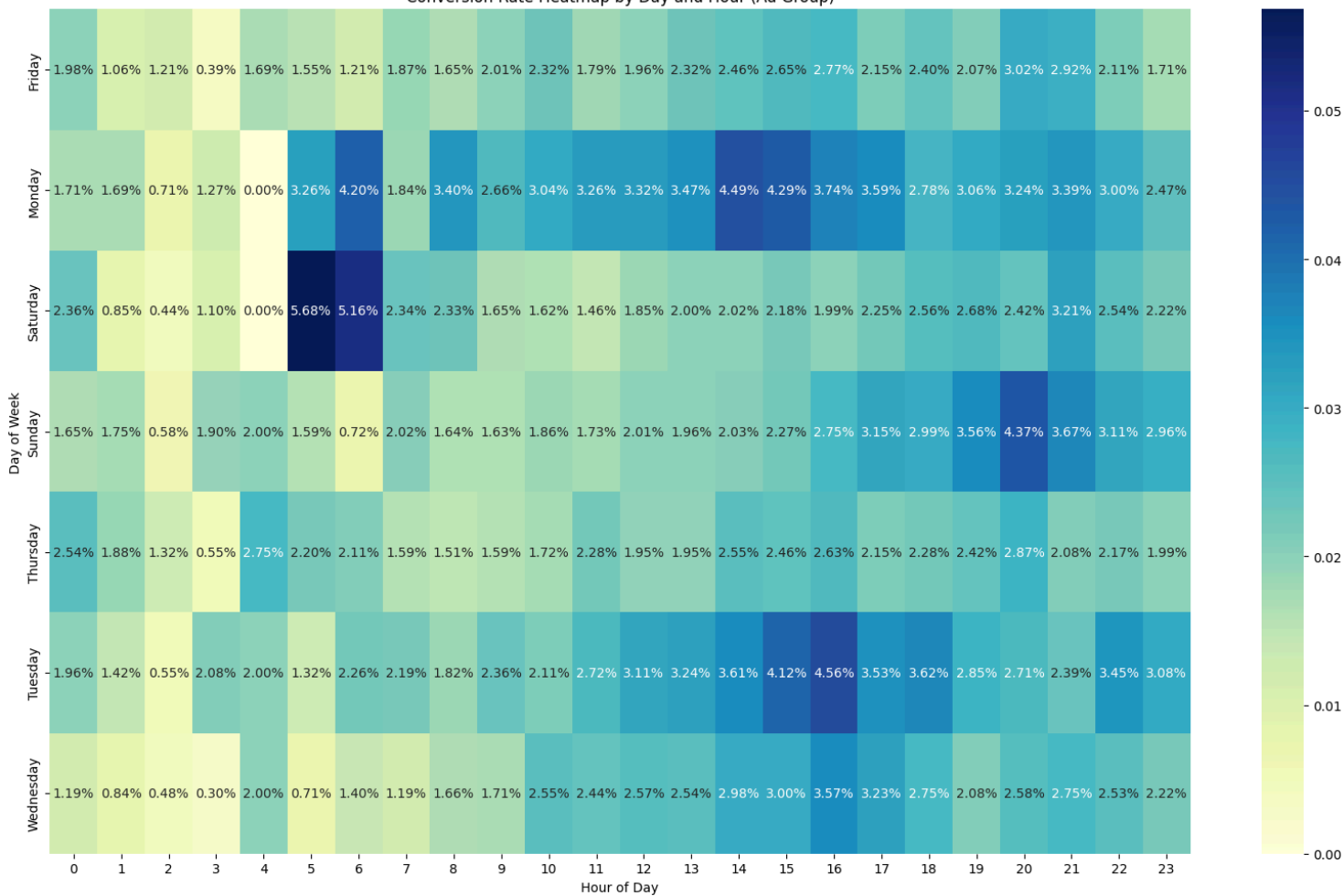
ROC AUC: 0.6318947321127621



<Figure size 640x480 with 0 Axes>



Conversion Rate Heatmap by Day and Hour (Ad Group)



In [10]: # A/B Testing Marketing Campaign – Precision Optimization

```
# 📦 Required Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, roc_auc_score, precision_recall_curve, brier_score_loss
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.calibration import CalibratedClassifierCV
from imblearn.over_sampling import SMOTE
import shap

# 📄 Load Dataset
df = pd.read_csv('marketing_AB.csv')

# 🛠 Feature Engineering
df['converted'] = df['converted'].astype(int)
df['evening_viewer'] = df['most ads hour'].apply(lambda x: 1 if 17 <= x <= 22 else 0)
df['weekend_viewer'] = df['most ads day'].isin(['Saturday', 'Sunday']).astype(int)
df['log_total_ads'] = np.log1p(df['total ads'])
df['ads_bin'] = pd.cut(df['total ads'], bins=[0, 50, 150, np.inf], labels=['low', 'medium', 'high'])

# Encode ads_bin
df = pd.get_dummies(df, columns=['ads_bin'], drop_first=True)

# Dummies for categorical features
df = pd.get_dummies(df, columns=['test group', 'most ads day'], drop_first=True)

# Feature selection
features = ['log_total_ads', 'most ads hour', 'evening_viewer', 'weekend_viewer'] + [col for col in df.columns if col.startswith('test group_') or col.startswith('most ads day_') or col.startswith('ads_bin_')]
X = df[features]
y = df['converted']

# ⚖ Balance Classes using SMOTE
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, random_state=42)
sm = SMOTE(random_state=42)
X_resampled, y_resampled = sm.fit_resample(X_train, y_train)

# 🚀 Train XGBoost with scale_pos_weight
xgb = XGBClassifier(n_estimators=100, use_label_encoder=False, eval_metric='logloss',
                    scale_pos_weight=(len(y_resampled) - sum(y_resampled)) / sum(y_resampled))
xgb.fit(X_resampled, y_resampled)

# 📊 Calibrate Model with Isotonic Regression
calibrated_xgb = CalibratedClassifierCV(xgb, method='isotonic', cv=3)
calibrated_xgb.fit(X_resampled, y_resampled)

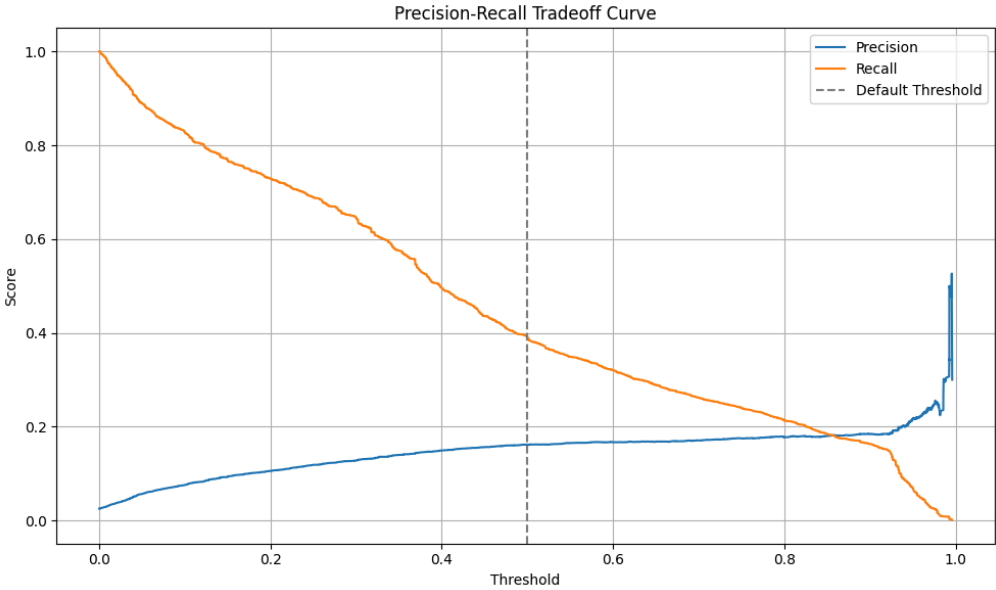
# 📈 Precision-Recall Threshold Curve
y_probs = calibrated_xgb.predict_proba(X_test)[:, 1]
precisions, recalls, thresholds = precision_recall_curve(y_test, y_probs)

plt.figure(figsize=(10, 6))
plt.plot(thresholds, precisions[:-1], label='Precision')
plt.plot(thresholds, recalls[:-1], label='Recall')
plt.axvline(x=0.5, color='gray', linestyle='--', label='Default Threshold')
plt.xlabel('Threshold')
plt.ylabel('Score')
plt.title('Precision-Recall Tradeoff Curve')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# 📋 Classification Report (Threshold Tuned Example)
optimal_threshold = 0.75
y_pred_opt = (y_probs >= optimal_threshold).astype(int)
```

```
print(f"Classification Report at Threshold {optimal_threshold}")
print(classification_report(y_test, y_pred_opt))
print("ROC AUC:", roc_auc_score(y_test, y_probs))
print("Brier Score Loss (Calibration):", brier_score_loss(y_test, y_probs))

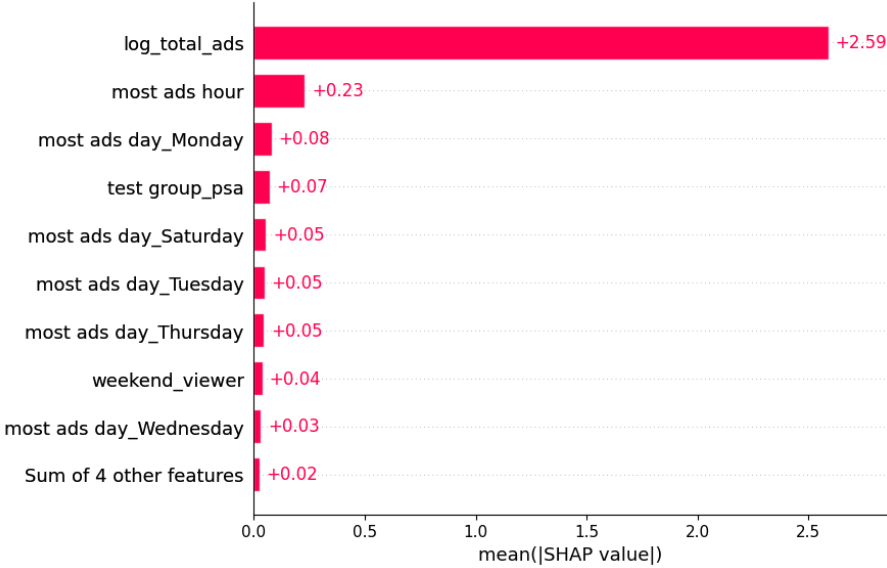
# SHAP Analysis
explainer = shap.Explainer(xgb)
shap_values = explainer(X_test[:1000])
shap.plots.bar(shap_values)
```



Classification Report at Threshold 0.75

	precision	recall	f1-score	support
0	0.98	0.97	0.98	143315
1	0.18	0.24	0.20	3711
accuracy			0.95	147026
macro avg	0.58	0.61	0.59	147026
weighted avg	0.96	0.95	0.96	147026

ROC AUC: 0.8507939402638152
Brier Score Loss (Calibration): 0.056298787836338765



```
In [12]: # Campaign Segmentation + Scoring Dashboard + Optimization Rules (Patched)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import precision_recall_curve
from xgboost import XGBClassifier

# Load dataset
df = pd.read_csv('marketing_AB.csv')
df['converted'] = df['converted'].astype(int)

# Feature engineering
df['log_total_ads'] = np.log1p(df['total_ads'])
df['evening_viewer'] = df['most_ads_hour'].apply(lambda x: 1 if 17 <= x <= 22 else 0)
df['weekend_viewer'] = df['most_ads_day'].isin(['Saturday', 'Sunday']).astype(int)
df['exposure_level'] = pd.cut(df['total_ads'], bins=[0, 50, 150, np.inf], labels=['low', 'medium', 'high'])
df = pd.get_dummies(df, columns=['test_group', 'most_ads_day'], drop_first=True)
df = pd.get_dummies(df, columns=['exposure_level'], drop_first=True)

# Select features
features = ['log_total_ads', 'most_ads_hour', 'evening_viewer', 'weekend_viewer'] + [col for col in df.columns if col.startswith('test_group_') or col.startswith('most_ads')]
X = df[features]
y = df['converted']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, random_state=42)

# Train XGBoost
xgb = XGBClassifier(n_estimators=100, eval_metric='logloss', use_label_encoder=False, scale_pos_weight=37)
xgb.fit(X_train, y_train)

# Predict probabilities
y_probs = xgb.predict_proba(X_test)[:, 1]

# Precision-recall for each exposure level
```

```

X_test['prob'] = y_probs
X_test['actual'] = y_test.values
X_test['exposure'] = pd.cut(np.expm1(X_test['log_total_ads']), bins=[0, 50, 150, np.inf], labels=['low', 'medium', 'high'])

thresholds = {}
for level in ['low', 'medium', 'high']:
    segment = X_test[X_test['exposure'] == level]
    p, r, t = precision_recall_curve(segment['actual'], segment['prob'])
    f1 = 2 * p * r / (p + r + 1e-9)
    best_thresh = t[np.argmax(f1)]
    thresholds[level] = best_thresh

# Define optimization rule functions (Patched)
def optimize_campaign(user_row):
    exposure = 'low'
    if user_row['total_ads'] > 150:
        exposure = 'high'
    elif user_row['total_ads'] > 50:
        exposure = 'medium'

    user_features = user_row[features].values.reshape(1, -1)
    prob = xgb.predict_proba(user_features)[0, 1]

    if prob >= thresholds[exposure]:
        return 'Target'
    else:
        return 'Hold'

# Apply rule to a sample
df['campaign_action'] = df.apply(optimize_campaign, axis=1)

# Summary report
actions = df.groupby('campaign_action')['converted'].agg(['count', 'mean'])
print("\nCampaign Action Summary:\n", actions)

# Optional: Export to dashboard/CSV
df[['user_id', 'total_ads', 'most_ads_hour', 'campaign_action']].to_csv('campaign_decisions.csv', index=False)

```

Campaign Action Summary:

	count	mean
campaign_action		
Hold	543351	0.014398
Target	44750	0.156872